

# Mana Pantalu (మన పంటలు) — Prototype Document Pack

Source: `Mana_Pantalu.pptx` (11 slides). Everything here is derived from that deck; where the deck was silent I made a decision and marked it [ASSUMPTION] so you can change it.

## 1. What "prototype" means here (scope lock)

Goal: a phone app you can demo to farmers, mentors, judges or an agriculture officer where **one thing is real: photo -> diagnosis -> Telugu voice/text guidance**. Everything else can be light or mocked.

| Real in prototype                             | Mocked / light in prototype                                      |
|-----------------------------------------------|------------------------------------------------------------------|
| Camera/gallery photo -> AI diagnosis (Gemini) | Treatment content: small hand-written table for ~4 crops         |
| Telugu/English switch, Telugu voice playback  | Push notifications (local reminders only)                        |
| Live weather (Open-Meteo, free)               | Expert confirmation (button shows "coming soon" / WhatsApp link) |
| Save crops, scan history, feedback            | Fertilizer/irrigation logic (rule-based tips from a table)       |

## 2. Best tool to build it (my honest recommendation)

**Primary: Flutter + Supabase + Gemini API, built with an AI IDE (Google Antigravity or Cursor) reading this doc pack from a `/docs` folder.** Why: the app depends on camera, text-to-speech, speech input, local notifications and an AI call with an image. That is custom-code territory, and Flutter gives one Android APK you can install on a farmer's phone. Supabase gives auth, database, image storage and a server function (to hide the Gemini key) on a free tier.

| Tool                                  | Use it for                                                                                                            | Verdict                                                                                                                                 |
|---------------------------------------|-----------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------|
| <b>Antigravity / Cursor (Flutter)</b> | Main build, all phases                                                                                                | Best                                                                                                                                    |
| <b>Jules</b>                          | After the repo is on GitHub: async tasks like "add ARB Telugu strings", "write widget tests", "fix analyzer warnings" | Good helper, not the main builder                                                                                                       |
| <b>FlutterFlow</b>                    | Drag-drop screens if you don't want code                                                                              | Weak fit: voice, camera flow and the Edge Function need custom code, and code export/limits depend on your plan (check current pricing) |
| <b>Lovable / free web builders</b>    | A 1-day clickable web demo (PWA) if you need something to show <i>this week</i>                                       | Fine as a throwaway; browser camera/TTS in Telugu is less reliable than native                                                          |

If you must pick one path today: **Flutter + Supabase + Gemini**.

### 3. Step-by-step build process

#### Phase 0 — Prep (half day)

1. Create a GitHub repo (`mana-pantalu`). Create folder (`/docs`) and drop all files from this pack in it.
2. Create Supabase project. Get a Gemini API key from Google AI Studio.
3. Collect **30–50 real photos** of rice/cotton/chilli/groundnut leaves (ANGRAU/KVK contacts, farmers, or the public PlantVillage dataset for a start). You need these to test diagnosis honestly.

#### Phase 1 — Backend foundation (half day)

4. Run the SQL from (`04_Database_Schema_and_Mock_Data.md`) (schema, RLS, seed). Create Storage bucket (`scans`).
5. Enable **Anonymous sign-ins** in Supabase Auth (guest-first login, see doc 06).

#### Phase 2 — App shell (1 day)

6. Give the AI (`AGENTS.md`) (in doc 09) + Prompt A. Result: Flutter project, theme, routing, 5-tab bottom nav, English/Telugu switch, empty screens.

#### Phase 3 — Static screens with mock data (1–2 days)

7. Prompt B: Welcome, Home (6 tiles), Weather, Crop library, Tips, My Crops, Profile. Use seed data only. Review on a real phone: touch targets, font size, Telugu rendering.

#### Phase 4 — The core feature (2 days)

8. Prompt C: Diagnose flow UI with a **fake** (`diagnose`) response so the whole flow works offline.
9. Deploy Edge Function (`diagnose`) (Prompt D)

and switch the app from fake to real. Test with your 30–50 photos; record how often it's right. Tune the prompt and the confidence threshold.

**Phase 5 — Voice, reminders, feedback (1 day)** 10. Prompt E: Telugu TTS playback on results and tips; voice-note feedback; local reminders for My Crops.

**Phase 6 — Test and ship (1–2 days)** 11. Build APK, install on 2–3 low-end Android phones, and watch 3–5 real farmers use it **without helping**. Note where they hesitate. Fix the top 5 problems. That feedback is worth more than any extra feature.

#### 4. Getting more out of it (beyond code)

- **Accuracy is the make-or-break.** Field photos (bad light, multiple leaves, mixed problems) are much harder than clean dataset photos. Measure it and show the number.
- **Don't let the AI invent treatments.** This pack uses a hybrid: Gemini only *identifies* crop + disease from a fixed list (or says "unknown"); the *treatment text* comes from a table your agriculture expert verifies. This is the difference between a demo and something safe to put in front of farmers.
- **Get one expert.** A local Krishi Vigyan Kendra (KVK) scientist, ANGRAU extension officer or Rythu Bharosa Kendra staff member to verify content. They are also your best pilot channel.
- **Telugu copy must be reviewed by a native, farm-literate speaker.** Telugu strings in this pack are drafts.
- **Be honest in the UI:** "possible issue", confidence shown, low confidence -> "ask an expert". The deck already says this; keep it.

## 5. Document map (answers to your list)

| #  | File                                                     | What it is                                                   |
|----|----------------------------------------------------------|--------------------------------------------------------------|
| 01 | <code>01_PRD.md</code>                                   | Product Requirements                                         |
| 02 | <code>02_Design_Document.md</code>                       | Visual design system + UX principles                         |
| 03 | <code>03_Tech_Stack.md</code>                            | Stack and why                                                |
| 04 | <code>04_Database_Schema_and_Mock_Data.md</code>         | SQL schema, RLS, seed data                                   |
| 05 | <code>05_UI_Wireframes_and_Component_Hierarchy.md</code> | Screens, ASCII wireframes, widget tree                       |
| 06 | <code>06_Auth_and_Route_Permissions.md</code>            | Auth model, routes, RLS matrix                               |
| 07 | <code>07_API_Contracts_and_Env_Vars.md</code>            | Endpoints, JSON contracts, env vars                          |
| 08 | <code>08_State_Management_and_User_Flows.md</code>       | Riverpod providers, flows, state machines                    |
| 09 | <code>09_AI_Prompts.md</code>                            | AGENTS.md, build prompts A-E, Gemini system prompt           |
| —  | <code>MASTER_SPEC.md</code>                              | All of the above in one file, for tools with one context box |

**How to feed the AI:** put the files in `/docs`, tell the AI to read them first, then give it **one phase prompt at a time** (doc 09). Don't paste everything and say "build the app" — results get worse the bigger the single ask.